

一种离散时隙上的周期报文 Offset 均衡分配方法

篠見由紀

2026 年 7 月

摘要

在汽车 CAN FD 网络中，多条周期报文若具有相同或高度聚集的首次发送延迟（Offset），其周期释放时刻会在超周期内反复重合，从而形成局部工作负载峰值。本文针对“仅调整周期报文初始 Offset，使报文释放在时间轴上尽量均匀”这一工程问题，建立离散时隙上的加权负载均衡模型。模型将 CAN FD 报文最坏传输时间作为负载权重，将 Offset 限定为有限离散集合，并以 5 ms 时隙峰值、超限程度、稳态离散度、启动阶段峰值和同时释放数量构成词典序目标。

在求解方法上，本文提出 GCLS（Greedy Construction and Conflict-directed Local Search）算法。该算法先依据周期和帧权重进行贪心构造，再通过单报文重定位、冲突导向双报文操作及随机重启改善解质量。为避免仅以启发式结果自证有效，本文同时给出原始配置、随机分配、纯贪心三类工程基线，并构建基于 CP-SAT 的整数优化模型作为小规模最优性对照。验证方案包括穷举校验、CP-SAT 最优性间隙、不同负载分布的重复实验、随机种子稳定性分析及 Vector/CANoe 结果对照。

本文以周期报文释放分布为直接研究对象，形成从启发式求解、精确对照到真实项目测量对照的完整验证闭环。所得结果用于评价 Offset 对局部峰值与整体均匀度的改善效果。

关键词： CAN FD；周期报文；Offset；负载均衡；贪心算法；局部搜索；CP-SAT

目录

1 引言	4
1.1 周期通信局部释放聚集的形成机制与相位配置	4
1.2 Offset 均衡与软件周期调度代价的耦合机制	4
1.3 研究方法与主要贡献	5
1.4 相关研究	6

1.4.1	CAN 可调度性分析、Offset 分配与流量整形	6
1.4.2	AUTOSAR COM 定时机制与周期调度组织	7
1.4.3	多目标启发式优化、Epsilon-constraint 与 Pareto 分析	7
2	问题边界与基本假设	8
2.1	研究边界	8
2.2	符号与输入	8
2.3	CAN FD 报文权重	9
2.4	平均负载可行性检查	9
3	离散时隙负载模型	9
3.1	超周期与时隙	9
3.2	启动阶段与稳态阶段	10
3.3	时隙负载与释放计数	10
3.4	5 ms 峰值约束	11
4	优化目标	11
4.1	峰值与超限量	11
4.2	均匀度指标	11
4.3	词典序目标	12
5	主算法: GCLS	12
5.1	总体结构	12
5.2	候选时隙预计算	12
5.3	贪心构造	12
5.4	单报文重定位局部搜索	13
5.5	冲突导向双报文搜索	13
5.6	随机重启	15
6	对照算法	16
6.1	原始 Offset 配置	16
6.2	随机分配	16
6.3	纯贪心	16
6.4	CP-SAT 精确对照	16
7	复杂度分析	18

8	正确性与不变量	19
9	实验设计与验证方法	19
9.1	对照组	19
9.2	主要评价指标	20
9.3	验证层次	20
9.3.1	单元级数学一致性	20
9.3.2	穷举验证	21
9.3.3	CP-SAT 对照	21
9.3.4	真实项目对照	21
9.4	测试集设计	22
10	示例说明	22
11	实现方法	22
12	适用范围与结果解释	23
13	结论	24

1 引言

1.1 周期通信局部释放聚集的形成机制与相位配置

车载通信系统中的控制量、状态量与诊断状态往往按照预先配置的周期重复发送。对于周期报文而言，周期决定相邻两次释放之间的时间间隔，Offset 则决定该周期序列相对于共同时间原点的初始相位。当多条报文具有相同或相近的 Offset，或者其周期关系使若干释放序列在公共倍数处反复相交时，原本分散在不同报文上的工作量会在局部时间窗口内聚集。以 CAN 或 CAN FD 为例，物理总线在同一时刻只能完成一帧传输；多个发送请求的同时出现不会增加总线的并行服务能力，而会形成仲裁、排队与短时需求峰值。因而，即使网络的长期平均工作量处于可接受范围，局部释放聚集仍可能放大瞬时竞争，并使仅依据平均负载作出的判断掩盖时间分布上的不均衡。

经典 CAN schedulability analysis 以优先级仲裁、阻塞与干扰为基础分析报文最坏响应时间 [3]；在此基础上，显式考虑 Offset 的分析与配置研究表明，对周期流进行去同步化能够改变干扰结构，并可在高负载条件下收紧响应时间估计或改善调度表现 [2, 1]。这些工作说明，相位配置不是无关紧要的实现细节，而是周期通信设计中的有效自由度。然而，在工程配置中，Offset 往往只能从给定的有限栅格中选取，报文的帧长和传输时间也并不相同。此时，“把报文错开”并不能充分刻画问题：需要明确局部窗口的划分方式、不同释放事件的权重、启动阶段与稳态阶段的区别，以及峰值约束和整体均衡性之间的优先关系，才能把直观的去同步要求转化为可计算、可审计的离散优化问题。

Offset 对长期平均释放工作量与局部分布的影响还必须严格区分。对于周期和单次释放权重固定的报文，改变 Offset 只平移释放序列，并不改变单位时间内的释放次数；在完整周期窗口或无限时域意义下，总工作量仍由各报文的周期与权重决定。因此，Offset 优化不能降低周期集合的长期平均释放工作量，也不能替代对平均负载不可行性的处理。其作用在于重塑工作量在时间轴上的局部分布，使峰值、超限程度与负载离散度在给定候选集合内得到改善。本文所研究的正是这一“总量守恒而局部分布可调”的组合优化问题。相应地，离散时隙负载是释放聚集程度的模型量，而不是完整 CAN 仲裁过程、端到端响应时间或实际总线利用率的同义表达。

1.2 Offset 均衡与软件周期调度代价的耦合机制

若只从总线时间轴观察，Offset 容易被理解为近乎“免费”的相位变量：只要周期、标识符和帧长不变，似乎可以在候选集合内任意移动释放序列，以换取更均匀的 CAN 侧分布。该理解忽略了周期报文通常还要由 ECU 软件中的周期入口组织和触发。以 AUTOSAR COM 为例，规范给出了 Com_MainFunctionTx 及其 ComMainTxTimeBase 等配置概念，但内部 timing handling 允许存在实现相关性 [6]。当项目采用共同整数时间原点和精确 tick 对齐的周期调度模型时，一条周期为 T_i 、Offset 为 O_i 的报文若由基本节拍 B 的周期入口精确表示，需要满足 $B \mid T_i$ 且 $B \mid O_i$ 。于是，该报文可支持的

最大基本节拍为

$$D_i = \gcd(T_i, O_i). \quad (1)$$

式 (1) 揭示了本文主线中的关键耦合：Offset 不仅决定周期报文在总线时间轴上的释放相位，还会改变软件侧周期调度能够采用的时间粒度。某个有利于错开 CAN 释放的 Offset，可能使 D_i 变小，迫使相关周期入口以更细的 TimeBase 被调用；反之，具有较大公约数的 Offset 可能有利于软件调度组织，却未必产生最佳的总线侧均衡。

上述整除关系是本文为配置建议建立的工程模型，而不是 AUTOSAR 规范对所有实现施加的唯一约束。在这一模型内，给定一个 MainFunction 分组后，其最大共同 TimeBase 由组内 D_i 的最大公约数确定，因而 TimeBase 无需再作为独立外层变量搜索。软件侧代价也不能在缺少目标 ECU 执行时间测量时直接称为 CPU utilization。本文以“每次调用的固定开销加组内报文的线性增量开销”为结构假设，构造

$$P_{\text{CPU}} = \sum_g \frac{\rho + N_g}{B_g}, \quad (2)$$

其中 N_g 和 B_g 分别表示组内报文数与共同 TimeBase， ρ 表示固定调用开销相对于单报文增量开销的比值。式 (2) 仅是比较不同分组结构的 CPU Cost Proxy，不是实际 CPU 占用率或百分比；其工程解释依赖于 ρ 的选取，并有待目标平台 profiling 标定。

由此，Offset 分配形成两个相互关联而又不能相互替代的目标：总线侧希望降低稳态离散时隙负载的聚集程度，软件侧希望在精确表示释放相位的前提下避免过细、过多的周期调用。若将 Offset、MainFunction 归属和 TimeBase 同时放入一个外层组合搜索，变量之间的笛卡尔积会迅速扩大，也会掩盖子问题自身的结构。本文采用分层处理：对每个固定 Offset assignment，以确定性 subset dynamic programming 精确求解式 (2) 下的 MainFunction partition；在外层，则继续以 GCLS 及其锚点、预算扫描和 refinement 搜索 Offset。这里的“exact”严格限定于固定 Offset 下的内层分组子问题，外层 Offset 搜索仍为 heuristic，二者不能混称为联合问题的全局精确求解。

在多目标层面，本文以 Peak 条件作为 CAN 侧 guardrail，并采用 ε -constraint 将 P_{CPU} 作为预算约束，在不同预算下搜索较小的稳态负载平方和。所有搜索阶段发现的候选被汇入累计 archive，再在 CAN 均衡指标与 CPU Cost Proxy 两个维度上进行非支配过滤。因此，所得集合是有限搜索过程支持下的 observed Pareto front，而非全局 Pareto frontier。几何 knee 只对最终 observed front 进行解释性后处理，用于给出一个边际代价变化较为明显的折中候选；它不参与搜索，也不构成唯一工程最优的证明。这样的限定使总线均衡、软件调度代价和推荐机制处于同一因果链中，同时避免把代理量、启发式搜索结果或几何解释外推为超出证据范围的结论。

1.3 研究方法 with 主要贡献

围绕“周期释放聚集如何在离散 Offset 约束下得到缓解，以及这种相位调整将付出何种软件调度代价”这一问题，本文建立从单目标均衡到联合折中的分层方法。主要贡献概括如下。

1. 建立面向周期报文释放聚集的离散时隙加权模型，将局部 Peak、物理阈值违反和稳态负载平方和置于统一评价框架，并明确证明 Offset 不改变长期平均释放工作量、只改变其局部分布。该模型解决了平均工作量与局部聚集容易混淆的问题；其结论限定于给定周期、候选 Offset、评价窗口和权重模型，不等同于完整 CAN schedulability 或端到端时延证明。
2. 提出以 greedy construction、精确增量 1-opt、conflict-directed pair search 和受控 restart 为核心的 GCLS，用于在有限 Offset 组合空间中求取高质量均衡解。该方法针对直接枚举难以扩展而单次贪心又容易受早期决策限制的问题，保留可复现的比较规则与审计信息；但 GCLS 及其外层候选搜索属于 heuristic，除非另有穷举或精确求解证书，本文不据此宣称 Offset assignment 全局最优。
3. 建立 Offset 与 MainFunction 调度粒度之间的最大公约数模型，并在固定 Offset 下将软件侧分组化为 D_i 类型集合上的确定性 subset-DP。该分层利用相同 D_i 类型可整体处理以及分组 TimeBase 由公约数唯一确定的结构，解决了同时搜索 Offset、分组和 TimeBase 所带来的组合膨胀；在式 (2) 及共同整数时间栅格等假设下，MainFunction partition 子问题是 exact，相关枚举对照用于核验实现一致性，但该精确性不扩展到外层 Offset 搜索，也不把 CPU Cost Proxy 转化为真实 CPU utilization。
4. 构造带 Peak guardrail 的 CAN-CPU ε -constraint 联合搜索，并以累计 archive、iterative refinement、二维非支配过滤和几何 knee 形成从候选发现到折中解释的闭环。该方法避免用缺少量纲解释的固定加权和掩盖两类目标，并允许显式观察不同 CPU 预算下的 CAN 均衡代价；其输出严格称为 observed Pareto front，锚点、refinement 与前沿覆盖均受外层 heuristic 和搜索预算限制，knee 仅是条件化、可解释的推荐候选而非唯一工程最优。

本文不在引言中预先使用尚未完成的九网段最终数值支撑上述贡献。后文将通过真实网段实验、内层精确求解器的独立枚举对照以及 seed、 ε 网格、 ρ 和 refinement 状态的敏感性分析，分别检验模型、算法与观察性结论；数值结果的外推范围以对应实验输入和搜索配置为限。

1.4 相关研究

1.4.1 CAN 可调度性分析、Offset 分配与流量整形

CAN 实时性研究的基础问题，是在固定优先级非抢占传输、阻塞和高优先级干扰作用下给出报文最坏响应时间上界。Tindell 等的分析框架奠定了经典 CAN response-time analysis 的基本形式 [3]。该类方法回答的是“给定报文参数和仲裁关系后是否满足时限”，与本文关注局部释放聚集具有直接联系，因为多个周期流的同步释放会形成保守但关键的干扰场景；但 schedulability analysis 的主要输出是响应时间或可调度性判据，并不等同于在有限 Offset 栅格上寻找加权释放分布较均匀的配置。

Offset-aware analysis 和 traffic shaping 工作进一步把相位信息纳入 CAN 研究。Grenier 等讨论了通过 Offset 去同步周期流以改善最坏响应时间的配置价值 [2]; Yomsi 等则在含 Offset 的事务模型上扩展响应时间分析, 使相位关系能够进入最坏情况推导 [1]。这些研究与本文共享“同步释放并非不可改变”的基本认识, 也为把 Offset 视为设计变量提供了依据。其重点通常落在响应时间分析、干扰界或 Offset 带来的性能增益, 而本文进一步关心离散候选、异质释放权重、局部 Peak guardrail 与整体平方和均衡之间的可计算关系, 并把 Offset 的软件调度代价纳入同一模型。因此, 本文方法不是对 CAN schedulability analysis 的替代, 而是面向配置生成的前置 traffic/load shaping: 它给出候选相位及其局部释放分布, 完整的响应时间和仲裁安全性仍需由相应分析或外部验证承担。

1.4.2 AUTOSAR COM 定时机制与周期调度组织

AUTOSAR COM 规范描述了 Com_MainFunctionTx、ComMainTxTimeBase 以及通信定时处理所需的配置语义 [6], 为周期发送在 ECU 软件侧的组织提供了标准化接口。另一方面, 周期任务研究通常以执行成本与调用周期之比刻画处理器需求 [7], 这一基本形式说明: 当等价功能需要更高调用频率时, 固定调度开销会被更频繁地支付。二者共同构成本文 CPU Cost Proxy 的建模背景, 但规范本身并不为给定一组报文自动求解 Offset、MainFunction partition 与 TimeBase 的联合配置, 实际 timing handling 也可能随生成器和平台实现而变化。

现有工程实践往往分别完成通信矩阵的 Offset 配置和 ECU 周期入口组织, 因而容易把总线相位与软件调度粒度视为两个串行、彼此独立的步骤。本文的定位是在明确共同整数时间栅格和 exact tick alignment 假设下, 揭示二者通过 $\gcd(T_i, O_i)$ 发生的结构耦合, 并对固定 Offset 下的分组给出精确组合求解。该模型用于生成可解释的配置建议, 不声称是 AUTOSAR 标准规定的唯一调度方式, 也不从配置结构直接推断实际 ECU CPU utilization。关于不同 AUTOSAR COM generator 的周期任务合并策略、实测调用开销模型及硬件平台 WCET 标定, 仍需补充更有针对性的公开研究或厂商资料 (TODO: 补充 COM MainFunction 周期组织与执行开销测量文献)。

1.4.3 多目标启发式优化、Epsilon-constraint 与 Pareto 分析

离散 Offset assignment 具有有限但快速增长的组合空间, 工程规模下常需借助贪心构造、局部搜索和多次 restart 等 heuristic 在可接受时间内寻找候选。此类方法能够利用问题结构并形成 anytime 式改进过程, 但若没有可采纳下界或完整枚举, 其结果只能被解释为已搜索空间中的高质量解。本文的 GCLS 属于这一范畴; 固定 Offset 下的 exact MainFunction 子求解器改善了每个候选的评价精度, 却不会自动赋予外层搜索全局最优性。

对于 CAN 均衡与 CPU Cost Proxy 这两个量纲和尺度不同的目标, 固定线性加权和需要预先给定难以解释的权重, 并可能遗漏离散非凸前沿上的部分解。 ε -constraint

方法保留一个目标作为优化量、把另一个目标转化为显式预算，是多目标数学规划中常用的后验解集生成思路 [8]。本文沿用这一思想，但每个预算下调用的是启发式 Offset 搜索，而非对完整联合空间的精确数学规划求解。因此，预算网格、初始锚点、搜索顺序与 refinement 共同决定候选覆盖范围，非支配过滤只能保证 archive 内的集合性质，输出必须称为 observed Pareto front。

Pareto 方法保留不可相互支配的候选，把偏好选择推迟到搜索之后；knee point 则常被用来识别一个目标继续改善时另一目标代价开始加速上升的区域 [9]。本文在最终 observed front 的归一化几何上采用 chord-based knee score，并为直线型前沿、点数不足等退化情况保留显式处理。其作用是减少工程人员浏览候选集的成本，而不是把偏好判断隐藏为新的优化目标。由于前沿会随 seed、 ε 分辨率、 ρ 和 refinement 预算变化，knee 也只能作为附带搜索条件的解释性折中推荐。面向车载周期通信的多目标 Offset 配置、尤其是同时涉及总线释放均衡与 COM 周期入口组织的公开工作仍需继续补充检索（TODO：补充车载通信多目标 Offset/任务组织联合优化文献）。

2 问题边界与基本假设

2.1 研究边界

本文仅考虑 DBC 中定义的周期 CAN FD 报文，不考虑事件报文、诊断报文、网络管理报文及其他偶发流量。算法只计算推荐 Offset，不修改报文 ID、周期、DLC、发送节点和总线速率，也不负责回写 DBC 或 ARXML。

本文直接优化 Offset 所决定的报文释放时刻，以离散时隙内的加权释放负载描述局部聚集程度。该建模方式保持了问题边界清晰、目标可解释和求解规模可控。

2.2 符号与输入

设周期报文集合为

$$\mathcal{M} = \{m_1, m_2, \dots, m_N\}. \quad (3)$$

每条报文 m_i 由下列参数描述：

$$m_i = (T_i, C_i, O_i, \mathcal{A}_i), \quad (4)$$

其中：

- T_i ：报文周期，单位为 ms；
- C_i ：报文最坏总线占用时间，单位统一换算为 μs ；
- O_i ：待求的首次发送延迟；
- $\mathcal{A}_i$ ：报文允许选择的 Offset 集合。

当前工程约束为

$$\mathcal{A}_i = \{15, 20, 25, \dots, 100\} \text{ ms}, \quad \forall i. \quad (5)$$

报文的第 k 个实例释放时刻为

$$r_{i,k} = O_i + kT_i, \quad k = 0, 1, 2, \dots \quad (6)$$

2.3 CAN FD 报文权重

为避免把 8 字节帧与 64 字节帧简单视作相同负载，本文以最坏总线占用时间 C_i 作为报文权重。若启用 BRS，则可抽象表示为

$$C_i = \frac{N_i^{\text{nom}}}{R_{\text{nom}}} + \frac{N_i^{\text{data}}}{R_{\text{data}}}, \quad (7)$$

其中 N_i^{nom} 和 N_i^{data} 分别为考虑最坏位填充后的仲裁相位与数据相位比特数； R_{nom} 、 R_{data} 分别为 nominal bitrate 与 data bitrate。若未启用 BRS，则全帧按 nominal bitrate 计算：

$$C_i = \frac{N_i^{\text{nom}} + N_i^{\text{data}}}{R_{\text{nom}}}. \quad (8)$$

CAN FD 的帧格式、BRS 相位切换和 DLC 映射可依据 Bosch M_CAN 文档实现 [4]。在算法层， C_i 被视为已由独立帧时长计算模块得到的整数微秒值。

2.4 平均负载可行性检查

长期平均总线负载为

$$U_{\text{avg}} = \sum_{i=1}^N \frac{C_i}{T_i}, \quad (9)$$

其中 C_i 与 T_i 必须换算到相同时间单位。Offset 不改变单位时间内的发送次数，因此不会改变 U_{avg} 。

设项目平均负载上限为 $\beta = 0.75$ 。若

$$U_{\text{avg}} > \beta, \quad (10)$$

则仅靠 Offset 不可能把平均负载降到 75%，算法应直接给出不可满足报告，而不是继续搜索。

3 离散时隙负载模型

3.1 超周期与时隙

定义周期集合的最小公倍数为

$$H = \text{lcm}(T_1, T_2, \dots, T_N). \quad (11)$$

当周期集合为 $\{10, 20, 50, 100, 500\}$ ms 时, $H = 500$ ms。设时隙宽度为

$$\Delta = 5 \text{ ms}, \quad (12)$$

则一个稳态超周期包含

$$M = \frac{H}{\Delta} \quad (13)$$

个时隙。对于上述周期集合, $M = 100$ 。

3.2 启动阶段与稳态阶段

由于 Offset 是首次发送延迟, O_i 与 $O_i + nT_i$ 在稳态下可能具有相同相位, 但启动过程并不相同。为避免固定分析区间偏向更晚启动的报文, 本文分别定义:

$$I^{\text{st}} = [0, O_{\max}), \quad (14)$$

$$I^{\text{ss}} = [O_{\max}, O_{\max} + H), \quad (15)$$

其中

$$O_{\max} = \max_i \max \mathcal{A}_i. \quad (16)$$

稳态区间长度固定为一个完整超周期, 因此每条报文在 I^{ss} 内恰好释放 H/T_i 次, 与 Offset 的绝对大小无关; 启动区间单独用于评价首次启动时的峰值。

定义报文 i 在选择 Offset o 时命中稳态时隙 s 的指示量:

$$a_{i,o,s}^{\text{ss}} = \begin{cases} 1, & \exists k : o + kT_i \in I_s^{\text{ss}}, \\ 0, & \text{其他}, \end{cases} \quad (17)$$

其中 I_s^{ss} 表示稳态区间中的第 s 个 5 ms 时隙。类似地定义启动阶段指示量 $a_{i,o,s}^{\text{st}}$ 。

3.3 时隙负载与释放计数

给定 Offset 向量

$$\mathbf{O} = (O_1, O_2, \dots, O_N), \quad (18)$$

稳态时隙 s 的加权负载为

$$L_s^{\text{ss}}(\mathbf{O}) = \sum_{i=1}^N C_i a_{i,O_i,s}^{\text{ss}}. \quad (19)$$

相应的同时释放报文数量为

$$K_s^{\text{ss}}(\mathbf{O}) = \sum_{i=1}^N a_{i,O_i,s}^{\text{ss}}. \quad (20)$$

启动阶段的 L_s^{st} 与 K_s^{st} 定义相同。

3.4 5 ms 峰值约束

将 5 ms 时隙内的释放工作量近似视为该窗口中的局部总线需求，则项目峰值约束为

$$L_s^{\text{ss}}(\mathbf{O}) \leq B, \quad B = \beta \Delta = 0.75 \times 5 \text{ ms} = 3.75 \text{ ms}. \quad (21)$$

当 C_i 使用 μs 表示时， $B = 3750 \mu\text{s}$ 。本文以该约束衡量 5 ms 离散时隙内的加权释放负载，并在真实项目对照中同步统计 1 ms、5 ms 与 10 ms 窗口指标。

4 优化目标

4.1 峰值与超限量

定义稳态最大时隙负载：

$$Z^{\text{ss}}(\mathbf{O}) = \max_s L_s^{\text{ss}}(\mathbf{O}). \quad (22)$$

定义启动阶段最大时隙负载：

$$Z^{\text{st}}(\mathbf{O}) = \max_s L_s^{\text{st}}(\mathbf{O}). \quad (23)$$

若峰值硬约束暂时不可满足，则使用超限时隙数与总超限量衡量违反程度：

$$N_{\text{vio}}(\mathbf{O}) = \sum_s \mathbb{I}[L_s^{\text{ss}}(\mathbf{O}) > B], \quad (24)$$

$$V_{\text{vio}}(\mathbf{O}) = \sum_s \max(0, L_s^{\text{ss}}(\mathbf{O}) - B). \quad (25)$$

4.2 均匀度指标

稳态总工作量

$$W = \sum_{s=1}^M L_s^{\text{ss}} \quad (26)$$

对任意 Offset 分配均为常数。平均时隙负载为

$$\bar{L} = \frac{W}{M}. \quad (27)$$

本文采用缩放后的绝对离差作为均匀度指标：

$$D(\mathbf{O}) = \sum_{s=1}^M |ML_s^{\text{ss}}(\mathbf{O}) - W|. \quad (28)$$

使用式 (28) 而不直接使用浮点方差，有两个好处：一是保持整数运算；二是可直接在线性 CP-SAT 模型中通过辅助变量表达。

此外定义稳态最大同时释放数：

$$K_{\text{max}}(\mathbf{O}) = \max_s K_s^{\text{ss}}(\mathbf{O}). \quad (29)$$

4.3 词典序目标

最终采用下列词典序目标向量：

$$\mathbf{F}(\mathbf{O}) = (N_{\text{vio}}, V_{\text{vio}}, Z^{\text{ss}}, Z^{\text{st}}, D, K_{\text{max}}). \quad (30)$$

两个解的优劣按照从左到右的顺序比较。也就是说：

1. 优先减少违反 75% 峰值约束的时隙数量；
2. 再减少总超限量；
3. 在可行解中优先降低稳态最大峰值；
4. 再降低启动阶段峰值；
5. 再使整体负载更加均匀；
6. 最后减少同一时隙内的报文释放数量。

词典序目标避免了人为设置加权系数。例如，使用简单加权和时，均匀度的改善可能错误地抵消峰值恶化；词典序目标不会出现这种情况。

5 主算法：GCLS

5.1 总体结构

GCLS 是 Greedy Construction and Conflict-directed Local Search 的缩写。算法流程如 fig. 1 所示。

5.2 候选时隙预计算

对每条报文 i 、每个候选 Offset $o \in \mathcal{A}_i$ ，预先计算命中稳态与启动时隙集合：

$$\mathcal{S}_{i,o}^{\text{ss}} = \{s \mid a_{i,o,s}^{\text{ss}} = 1\}, \quad (31)$$

$$\mathcal{S}_{i,o}^{\text{st}} = \{s \mid a_{i,o,s}^{\text{st}} = 1\}. \quad (32)$$

在试探某个候选 Offset 时，只需在相应集合中的时隙上加减 C_i ，无需重新展开全部时间轴。

5.3 贪心构造

贪心阶段先按以下键值排序报文：

$$(T_i, -C_i, \text{ID}_i). \quad (33)$$

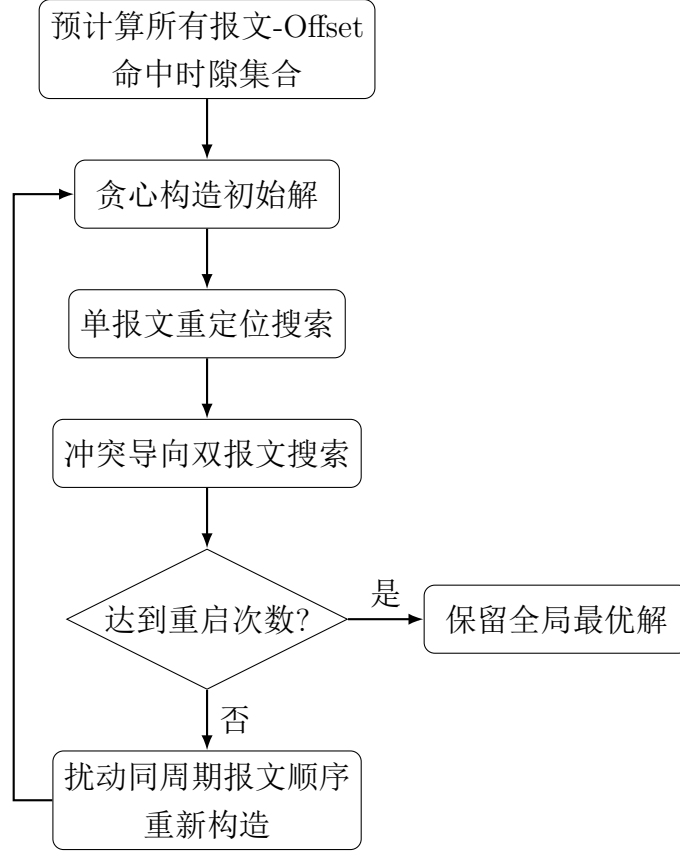


图 1: GCLS 算法流程

即周期越短越先处理；周期相同时，帧越长越先处理；仍相同时，以 CAN ID 保证确定性。短周期报文在超周期内出现次数更多，一旦位置选择不佳，对负载分布影响更大。

对每条报文，枚举全部候选 Offset，选择使当前部分解词典序目标最小的候选值。

5.4 单报文重定位局部搜索

纯贪心的主要缺点是早期决策不可回退。单报文重定位（1-opt relocation）固定其他报文不变，依次重新枚举每条报文的所有候选 Offset。若新位置使全局目标改善，则接受移动。循环执行，直到完整一轮没有改进。

该阶段结束时，任意单条报文独立改变 Offset 都不能继续改善词典序目标，因此得到 1-opt 局部最优解。

5.5 冲突导向双报文搜索

1-opt 搜索可能停在平台区：任何单条报文移动都会先恶化目标，但两条报文同时移动却可能改善整体分布。为此，GCLS 只在最拥挤时隙附近进行有限的双报文搜索，而不枚举全部报文对。

输入: 报文集合 $\mathcal{M}$, 候选 Offset 集合 $\mathcal{A}_i$, 预计算时隙集合

输出: 初始 Offset 向量 $\mathbf{O}$

按式 (33) 对报文排序;

初始化所有时隙负载为零;

```
foreach  $m_i \in \mathcal{M}$  do
   $o^* \leftarrow \emptyset, F^* \leftarrow +\infty;$ 
  foreach  $o \in \mathcal{A}_i$  do
    临时将  $m_i$  以 Offset  $o$  加入当前状态;
     $F \leftarrow \text{Evaluate}(\mathbf{O} \cup \{O_i = o\});$ 
    撤销临时加入;
    if  $F$  按词典序优于  $F^*$  then
       $F^* \leftarrow F, o^* \leftarrow o;$ 
    end
  end
  永久设置  $O_i \leftarrow o^*$  并更新时隙状态;
end
return  $\mathbf{O};$ 
```

输入: 贪心初始解 $\mathbf{O}$

输出: 1-opt 局部最优解

$improved \leftarrow true;$

```
while  $improved$  do
   $improved \leftarrow false;$ 
  foreach  $m_i \in \mathcal{M}$  do
    保存当前 Offset  $o_0 \leftarrow O_i;$ 
    从状态中移除  $m_i;$ 
    在  $\mathcal{A}_i$  中寻找使完整目标最优的  $o^*;$ 
    将  $m_i$  以  $o^*$  重新加入状态;
    if  $o^* \neq o_0$  且目标严格改善 then
       $O_i \leftarrow o^*, improved \leftarrow true;$ 
    end
    else
      恢复  $O_i \leftarrow o_0;$ 
    end
  end
end
return  $\mathbf{O};$ 
```

设负载最高的前 q 个稳态时隙集合为

$$\mathcal{H}_q = \text{TopQ}(\{L_s^{\text{ss}}\}_{s=1}^M). \quad (34)$$

定义冲突候选报文集合

$$\mathcal{C} = \{m_i \mid \exists s \in \mathcal{H}_q, a_{i,O_i,s}^{\text{ss}} = 1\}. \quad (35)$$

对 $\mathcal{C}$ 中的报文对 (m_i, m_j) , 尝试两类操作:

1. **双重定位**: O_i 与 O_j 同时移动到邻域候选;
2. **Offset 交换**: 交换 O_i 与 O_j , 前提是交换后的值分别属于对方候选集合。

邻域候选可限制为

$$\mathcal{N}(O_i) = \{O_i \pm 5, O_i \pm 10, O_i \pm 15\} \cap \mathcal{A}_i, \quad (36)$$

以控制计算量。

输入: 当前 1-opt 解 $\mathbf{O}$, 热点时隙数 q , 候选上限 c

输出: 改进后的 Offset 向量

选择负载最高的 q 个时隙, 构造冲突报文集合 $\mathcal{C}$;

按报文对热点负载的贡献排序, 仅保留前 c 条;

```

foreach  $(m_i, m_j) \in \mathcal{C} \times \mathcal{C}, i < j$  do
  foreach  $o_i \in \mathcal{N}(O_i)$  do
    foreach  $o_j \in \mathcal{N}(O_j)$  do
      试探双重定位  $(O_i, O_j) \leftarrow (o_i, o_j)$ ;
      if 目标严格改善 then
        接受并重新执行单报文局部搜索;
      end
    end
  end
  if 交换  $O_i, O_j$  合法且目标严格改善 then
    接受交换并重新执行单报文局部搜索;
  end
end
return  $\mathbf{O}$ ;

```

5.6 随机重启

贪心构造对报文排序敏感。为了降低这一敏感性, 执行 K 次随机重启。每次重启保持“周期升序、权重降序”的大类顺序, 仅对周期相同且权重接近的报文随机打乱。每次运行完整的“贪心 + 1-opt + 冲突双报文”流程, 并保留全局最优结果。

随机重启不是通过随机接受劣解来跳出局部最优，因此比模拟退火更容易复现和解释。本文默认取 $K = 20$ ，并记录全部随机种子。

6 对照算法

6.1 原始 Offset 配置

若项目 DBC 中已有 Offset，则直接以原始配置作为工程基线：

$$\mathbf{O}^{\text{orig}}. \quad (37)$$

若原始项目全部为 0，但 0 不属于新候选集合，则原始配置只用于展示优化前释放聚集，不作为约束内可行解。

6.2 随机分配

对每条报文独立均匀采样：

$$O_i \sim \text{Uniform}(\mathcal{A}_i). \quad (38)$$

重复 R 次，报告目标指标的中位数、最好值和最差值。随机分配用于衡量“随便错开”能达到的水平，并检验 GCLS 的改进是否来自算法本身，而不是仅仅来自 Offset 非零。

6.3 纯贪心

纯贪心采用 algorithm 1，不执行局部搜索。它是最重要的消融基线，用于量化局部求解器带来的收益：

$$\Delta Z = Z_{\text{greedy}} - Z_{\text{GCLS}}. \quad (39)$$

若 $\Delta Z = 0$ ，仍需比较均匀度 D 、热点时隙数量及启动峰值。

6.4 CP-SAT 精确对照

CP-SAT 适合处理离散整数决策，可返回 OPTIMAL、FEASIBLE、INFEASIBLE 或 UNKNOWN 等求解状态 [5]。本文用其建立小规模精确或限时近似最优基线。

对每条报文 i 和候选 Offset $o \in \mathcal{A}_i$ ，定义二元变量

$$x_{i,o} = \begin{cases} 1, & O_i = o, \\ 0, & \text{其他.} \end{cases} \quad (40)$$

唯一选择约束为

$$\sum_{o \in \mathcal{A}_i} x_{i,o} = 1, \quad \forall i. \quad (41)$$

稳态时隙负载为

$$L_s = \sum_{i=1}^N \sum_{o \in \mathcal{A}_i} C_i a_{i,o,s}^{\text{ss}} x_{i,o}. \quad (42)$$

引入最大负载变量 Z :

$$L_s \leq Z, \quad \forall s. \quad (43)$$

若要求验证 75% 峰值可行性, 再加入

$$L_s \leq B, \quad \forall s. \quad (44)$$

为表达绝对离差, 引入非负整数变量 d_s :

$$d_s \geq ML_s - W, \quad (45)$$

$$d_s \geq W - ML_s, \quad (46)$$

并定义

$$D = \sum_s d_s. \quad (47)$$

最大释放数量可由

$$K_s = \sum_i \sum_o a_{i,o,s}^{\text{ss}} x_{i,o}, \quad (48)$$

$$K_s \leq K_{\max} \quad (49)$$

表示。

为了保持词典序, 与其构造一个巨大加权和, 更稳妥的方式是分阶段求解:

1. 最小化 Z , 得到 Z^* ;
2. 加入 $Z = Z^*$, 最小化 D , 得到 D^* ;
3. 加入 $D = D^*$, 最小化 $K_{\max}$;
4. 如需考虑启动峰值, 可进一步固定前三阶段结果并最小化 Z^{st} 。

若硬约束模型不可行, 则移除式 (44), 最小化 Z , 求得当前网络在给定 Offset 集合下能达到的理论最低稳态峰值。

7 复杂度分析

设：

- N ：报文数量；
- $A = |\mathcal{A}_i|$ ：每条报文候选 Offset 数，当前为 18；
- $M = H/\Delta$ ：稳态时隙数量，当前典型值为 100；
- $r_i = H/T_i$ ：报文 i 在一个稳态超周期内的释放次数；
- K ：随机重启次数；
- I ：单报文局部搜索迭代轮数。

预计算命中时隙集合的复杂度为

$$O\left(\sum_{i=1}^N Ar_i\right). \quad (50)$$

若每次候选评估对所有 M 个时隙重新扫描，贪心构造复杂度为

$$O(NAM). \quad (51)$$

利用命中集合增量更新后，试探性加减负载的代价近似为 $O(r_i)$ ；由于当前 $M = 100$ 很小，评分阶段保留 $O(M)$ 全扫描更利于保证代码正确性。

单报文局部搜索每轮复杂度同样为

$$O(NAM), \quad (52)$$

总复杂度约为 $O(INAM)$ 。

设冲突候选集合上限为 c ，每条报文邻域候选数量为 b ，则双报文搜索最坏复杂度为

$$O(c^2b^2M). \quad (53)$$

GCLS 总体复杂度近似为

$$O(K[(I+1)NAM + c^2b^2M]). \quad (54)$$

在 $N \approx 20$ 、 $A = 18$ 、 $M = 100$ 、 $K \leq 30$ 的项目规模下，该计算量对 Python 实现通常很轻。CP-SAT 的最坏复杂度仍具有组合爆炸风险，因此其定位是小规模对照和限时验证，而不是唯一工程主算法。

8 正确性与不变量

虽然 GCLS 不能保证全局最优，但实现必须满足以下基本不变量。

命题 1 (Offset 合法性). 算法输出满足

$$O_i \in \mathcal{A}_i, \quad \forall i. \quad (55)$$

命题 2 (稳态释放次数守恒). 在区间 $I^{\text{ss}} = [O_{\max}, O_{\max} + H)$ 内，每条报文 i 的释放次数恒为 H/T_i ，与 O_i 无关。

证明. 区间长度为 H ，且 H 是 T_i 的整数倍。周期序列 $O_i + kT_i$ 在任意长度为 H 的半开区间内恰好出现 H/T_i 个元素。□

命题 3 (稳态总负载守恒). 对于任意合法 *Offset* 向量 $\mathbf{O}$ ，有

$$\sum_{s=1}^M L_s^{\text{ss}}(\mathbf{O}) = \sum_{i=1}^N \frac{H}{T_i} C_i = W. \quad (56)$$

该性质用于单元测试：任何 apply/rollback、局部移动或双报文操作后，稳态总负载都不得改变。

命题 4 (局部搜索单调性). 若局部搜索只接受严格改善词典序目标的操作，则算法产生的目标序列严格下降，并将在有限步内停止。

证明. 合法 Offset 组合数量为有限值 $\prod_i |\mathcal{A}_i|$ 。每次接受操作都使词典序目标严格下降，因此不会重复访问同一目标状态，也不可能无限执行。□

9 实验设计与验证方法

9.1 对照组

本文设置表 1 所示的五组对照方法。

表 1: 算法对照组

方法	作用	结果性质
Original	原始 Vector/DBC Offset 配置	工程现状基线
Random	独立随机选择 Offset，多次重复	随机错峰基线
Greedy	仅运行贪心构造	消融局部搜索收益
GCLS	贪心 + 1-opt + 冲突双报文 + 重启	主工程算法
CP-SAT	整数约束优化，限时或求最优	小规模最优性对照

9.2 主要评价指标

定义 5 ms 稳态峰值负载率:

$$P_5 = \frac{Z^{ss}}{\Delta}. \quad (57)$$

启动峰值负载率:

$$P_5^{st} = \frac{Z^{st}}{\Delta}. \quad (58)$$

均匀度可使用归一化绝对离差:

$$D_{\text{norm}} = \frac{1}{MW} \sum_s |ML_s - W|. \quad (59)$$

也可报告变异系数:

$$CV = \frac{\sqrt{\frac{1}{M} \sum_s (L_s - \bar{L})^2}}{\bar{L}}. \quad (60)$$

相对于原始配置的峰值改善率:

$$\eta_Z = \frac{Z_{\text{orig}} - Z_{\text{alg}}}{Z_{\text{orig}}} \times 100\%. \quad (61)$$

当 CP-SAT 求得最优峰值 Z^* 时, GCLS 的最优性间隙为

$$\text{Gap}_Z = \frac{Z_{\text{GCLS}} - Z^*}{\max(Z^*, 1)} \times 100\%. \quad (62)$$

若 CP-SAT 只在时限内得到下界 LB , 则报告

$$\text{Gap}_{LB} = \frac{Z_{\text{GCLS}} - LB}{\max(LB, 1)} \times 100\%. \quad (63)$$

本文同时报告:

- 超限时隙数 N_{vio} 与总超限量 V_{vio} ;
- 最大同时释放报文数 K_{max} ;
- 算法运行时间;
- GCLS 多个随机种子下的均值、标准差、最好值与最差值;
- 局部搜索相对于纯贪心的增量改善。

9.3 验证层次

9.3.1 单元级数学一致性

必须验证:

1. 预计算时隙集合与直接展开释放时刻的结果完全一致；
2. 增量 apply/rollback 后，全部负载数组和目标值恢复原状；
3. 稳态总释放次数和总负载满足守恒命题；
4. 所有输出 Offset 均属于合法集合；
5. 局部搜索接受操作后，词典序目标严格改善。

9.3.2 穷举验证

对于小规模实例，直接枚举所有 Offset 组合：

$$|\Omega| = A^N. \quad (64)$$

当 $A = 18$ 时，本文对 $N \leq 5$ 的实例执行完整穷举；对于 $N \leq 8$ 的实例，通过缩小候选 Offset 集合执行补充穷举。穷举结果用于确认：

- CP-SAT 模型与直接评价函数的一致性；
- GCLS 是否能找到全局最优或接近最优解；
- 词典序比较实现是否正确。

9.3.3 CP-SAT 对照

对 8–20 条报文的小中规模实例设置固定求解时限，例如 60 s 或 300 s。报告求解状态、最优目标、最佳下界和时限内 gap。若 CP-SAT 返回 OPTIMAL，则可直接评价 GCLS 的真实最优性间隙；若返回 FEASIBLE，则仅以其下界和当前最好解进行夹逼。

9.3.4 真实项目对照

将 GCLS 输出的 Offset 手动配置到 Vector 工具或等价仿真环境中，对比：

- 报文释放时刻图；
- 1 ms、5 ms、10 ms 滑动窗口真实总线占用率；
- 最大连续忙碌时间；
- 优化前后热点时间点及峰值位置的一致性；
- 离散模型与实测窗口负载的相对误差。

该对照用于量化离散释放负载模型与真实 CAN FD 项目测量结果之间的一致性 & 误差范围。

9.4 测试集设计

表 2: 实验数据集

数据集	报文数	周期特征	验证目标
四报文示例	4	10/20/50/100 ms	人工核对释放时刻与最优 Offset
小规模均衡集	6–8	周期较均匀	穷举与 CP-SAT 精确对照
高频拥挤集	12	10/20 ms 较多	检验峰值约束与冲突搜索
典型混合集	20	100 ms 最多, 10 ms 最少	主工程性能评价
大帧压力集	20	48/64 B 报文较多	检验加权模型优于报文计数
扩展集	50–100	保持典型周期集合	运行时间与稳定性分析

10 示例说明

考虑四条周期报文，周期分别为

$$(T_1, T_2, T_3, T_4) = (20, 100, 10, 50) \text{ ms.} \quad (65)$$

若四条报文均以 $O_i = 0$ 启动，则在 $t = 0, 100, 200, \dots$ ms 时四条报文同时释放。若所有帧权重相等，则这些时隙的释放计数为 4，其余部分时隙负载较低。

GCLS 会先处理 10 ms 周期报文，再处理 20 ms、50 ms 和 100 ms 报文。假设候选集合为式 (5)，算法可能得到类似

$$(O_1, O_2, O_3, O_4) = (20, 35, 15, 25) \text{ ms} \quad (66)$$

的候选解。该示例数值仅用于说明形式，最终结果必须由实际 C_i 、Offset 候选和词典序目标计算得到。

需要特别注意：对于 10 ms 周期报文，15 ms 与 25 ms 在稳态具有相同相位，但启动时刻不同。因此本文把稳态分布作为主优化对象，同时保留启动峰值作为次级目标。

11 实现方法

数学求解模块仅依赖统一数据模型，不直接读取 DBC 或 ARXML。输入层完成文件解析后，向优化器提供：

```
Message(
    name,
    cycle_time_us,
```

```
frame_time_us,  
allowed_offsets_us,  
original_offset_us,  
sender_ecu,  
)
```

核心优化器只需维护：

- `steady_slot_loads[s]`;
- `startup_slot_loads[s]`;
- `steady_slot_counts[s]`;
- 每个 (i, o) 的稳态/启动命中时隙集合;
- 当前 Offset 向量与词典序目标。

本文将主算法、对照算法与验证模块组织为：

```
optimizers/  
greedy.py  
local_search.py  
gcls.py  
cp_sat.py  
  
evaluation/  
metrics.py  
objective.py  
exhaustive.py
```

其中 `gcls.py` 构成默认求解流程，`cp_sat.py` 和 `exhaustive.py` 用于精确对照与回归测试。

12 适用范围与结果解释

本文模型优化的是周期报文释放工作量在离散时间轴上的分布。其评价结果直接反映 Offset 对峰值时隙、超限时隙数量、整体离散度、启动峰值及同时释放数量的影响。

模型的适用范围限定如下：

1. 5 ms 时隙模型描述时隙尺度上的释放负载，不刻画帧在时隙内部的精确连续位置；

2. 研究对象限于 DBC 中定义的周期 CAN FD 报文，不包含偶发流量、发送抖动、时钟漂移及错误重传；
3. 75% 为本项目采用的设计阈值，用于判定 5 ms 时隙负载是否满足工程约束；
4. 算法结论以 Offset 均衡分配效果为边界，不将离散负载指标外推为完整的端到端时延证明。

本文通过数学一致性测试、穷举、CP-SAT 对照和 Vector/CANoe 项目测量共同验证结论。四类验证使用同一 Offset 向量与统一评价口径，从而保证算法结果、精确模型和工程观测之间具有可比性。

13 结论

本文针对 CAN FD 周期报文初始 Offset 均衡分配问题，建立了边界明确的离散时隙优化模型，直接解决报文释放时刻错开与超周期局部负载均衡问题。

GCLS 以贪心构造提供高质量初始解，通过单报文重定位消除早期贪心决策的局限，再利用冲突导向双报文操作突破 1-opt 平台，并通过随机重启降低排序敏感性。纯贪心、随机分配与原始配置构成工程对照，CP-SAT 和穷举则提供小规模最优性验证。该组合兼顾可解释性、实现难度、运行速度和学术验证完整性，适合约 20 条乃至更大规模的典型周期报文集合。

参考文献

- [1] P. M. Yomsi, D. Bertrand, N. Navet, and R. I. Davis. Controller Area Network (CAN): Response Time Analysis with Offsets. In *Proceedings of the IEEE International Workshop on Factory Communication Systems (WFCS)*, 2012.
- [2] M. Grenier, L. Havet, and N. Navet. Pushing the Limits of CAN—Scheduling Frames with Offsets Provides a Major Performance Boost. In *Proceedings of the 4th European Congress Embedded Real Time Software*, 2008.
- [3] K. Tindell, H. Hansson, and A. Wellings. Analyzing Real-Time Communications: Controller Area Network (CAN). In *Proceedings of the 15th IEEE Real-Time Systems Symposium (RTSS)*, 1994.
- [4] Robert Bosch GmbH. *M_CAN User's Manual*, Revision 3.3.1. Available at: https://www.bosch-semiconductors.com/media/ip_modules/pdf_2/m_can/mcan_users_manual_v331.pdf.

- [5] Google Optimization Team. *CP-SAT Solver, OR-Tools Documentation*. Available at: https://developers.google.com/optimization/cp/cp_solver, accessed July 2026.
- [6] AUTOSAR. *Specification of Communication*, Classic Platform R25-11, Document ID 15, 2025. Available at: https://www.autosar.org/fileadmin/standards/R25-11/CP/AUTOSAR_CP_SWS_COM.pdf.
- [7] C. L. Liu and J. W. Layland. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, 20(1):46–61, 1973. DOI: <https://doi.org/10.1145/321738.321743>.
- [8] G. Mavrotas. Effective Implementation of the ε -Constraint Method in Multi-Objective Mathematical Programming Problems. *Applied Mathematics and Computation*, 213(2):455–465, 2009. DOI: <https://doi.org/10.1016/j.amc.2009.03.037>.
- [9] K. Deb and S. Gupta. Understanding Knee Points in Bicriteria Problems and Their Implications as Preferred Solution Principles. *Engineering Optimization*, 43(11):1175–1204, 2011. DOI: <https://doi.org/10.1080/0305215X.2010.548863>.